PodiumHQ — DevSecOps Capstone

Autonomous Hackathon & Event Judging Platform with Hardened GitOps Delivery

Core Problem Solved

- Replaces error-prone manual spreadsheets with an automated multi-criteria evaluation platform.
- Provides real-time weighted scoring across rounds with dynamic tie-breaking and rank calculation.
- Engineered specifically to solve high-concurrency judging deadlines with zero downtime.

DevSecOps Mission & Scope

- Demonstrates the full software delivery journey from local commit to cloud-native production.
- Enforces security gates: non-root containers, Trivy CVE scanning, and Gitleaks detection.
- Delivers declarative GitOps via ArgoCD, Helm 3 packaging, and Terraform-provisioned AWS EKS.

Project Metadata

- Author: Platform Engineering / DevSecOps Candidate
- Domain: Hackathon & Event Evaluation (Original Domain, Zero TaskBoard Overlap)
- Rubric Target: 100/100 Points across Modules M1 through M10

Application & Technology Stack

High-Performance Modern Web Architecture with Relational Integrity

Frontend Tier (React 18 SPA)

- React 18 + Vite with Tailwind CSS styling in an Obsidian & Solar Gold palette.
- Live leaderboard with instantaneous client-side ranking and category filtering.
- Responsive judge scoring form with client-side bounds and schema validation.
- Runtime: Served via unprivileged Nginx (UID 101) with dynamic proxying to backend.

Backend Tier (FastAPI Async API)

- Python 3.12 running asynchronous FastAPI ASGI server with Uvicorn.
- 15+ RESTful endpoints under /api/events, /api/teams, /api/judges, /api/scores, and /api/leaderboard.
- Operational probes: /health (liveness), /ready (database ping), and /metrics (Prometheus).
- Runtime: Hardened Python container running strictly as non-root appuser (UID 10001).

Data & Persistence Tier (PostgreSQL 16)

- PostgreSQL 16 relational database with Psycopg 3 connection pooling.
- SQLAlchemy 2.0 ORM with relational foreign keys, unique constraints, and indexes.
- Alembic schema migrations with automated pre-upgrade execution.
- Cloud Storage: AWS EBS gp3 volume claim via AWS EBS CSI driver.

End-to-End System Architecture

Complete Flow: Developer -> CI/CD -> GitOps -> AWS Cloud -> Observability

1. Developer	2. GitHub Actions	3. Registry	4. GitOps	5. AWS EKS	6. Monitoring
Git Push / Main	Lint -> Pytest 91% -> Trivy	GHCR (Git SHA Tag)	ArgoCD Auto-Sync	VPC + Ingress + HPA	Prometheus & Grafana

Pipeline Flow (DevSecOps Quality Gates)

- Pre-commit verification enforces Ruff formatting and Gitleaks secret scanning.
- GitHub Actions executes automated gates: linting, pytest coverage (>=80%), multi-stage builds.
- Trivy blocks image publishing if any HIGH or CRITICAL CVEs are detected.
- Approved container images are tagged with immutable Git commit SHAs—never :latest.

Cloud Runtime (AWS EKS & GitOps Delivery)

- ArgoCD polls the repository, detects image updates, and auto-syncs manifests to the cluster.
- Traffic flows: AWS ALB / Ingress Controller -> Frontend (:8080) -> Backend (:8000) -> PostgreSQL (:5432).
- Horizontal Pod Autoscaler (HPA) dynamically scales backend pods between 2 and 6 replicas based on CPU load.
- Prometheus scrapes application metrics every 15s via dedicated ServiceMonitor.

CI/CD Pipeline: GitHub Actions

Automated Quality Gates, Vulnerability Blocking, and Immutable Tagging

Gate 1: Code Quality & Static Analysis

- Ruff: Ultra-fast linting and code formatting checks across all backend Python modules.
- Mypy: Static type checking ensuring Pydantic and FastAPI schema integrity.
- ESLint: Static code analysis ensuring frontend React components follow strict standards.

Gate 2: Automated Testing & Coverage Gate

- Pytest: Executes 31 unit and integration tests against in-memory SQLite database.
- Coverage Gate: Pytest-cov enforces a hard 80% coverage threshold (currently achieving 91%).
- Pipeline Failure: Any failing test or coverage drop below 80% instantly terminates the workflow.

Gate 3: Hardened Build & Trivy Vulnerability Scan

- Docker Buildx: Multi-stage builds compiling frontend assets and backend dependencies.
- Trivy Scanner: Scans root filesystem and built images with exit-code: 1 on HIGH/CRITICAL CVEs.
- GHCR Publishing: Immutable tags using github.sha (e.g. ghcr.io/user/podiumhq-backend:f990516).

Infrastructure as Code: Terraform on AWS

Modular, Reproducible Cloud Infrastructure Provisioning across 3 AZs

VPC & Network Architecture (vpc.tf)

- Region: AWS ap-south-1 (Mumbai) across 3 Availability Zones for fault tolerance.
- Subnet Layout: 3 public subnets (for Ingress/ALBs) and 3 private subnets (for EKS worker nodes).
- Egress: Dedicated NAT Gateway in public subnet routing private outbound internet traffic.
- Kubernetes Tags: Strict kubernetes.io/role/elb and internal-elb tags for AWS Load Balancer Controller.

EKS Cluster & Node Groups (eks.tf)

- EKS Control Plane: Managed Kubernetes v1.30 control plane with private & public API endpoints.
- Managed Node Group: 2 to 4 t3.medium EC2 instances distributed across private subnets.
- IAM Roles: Least-privilege roles for EKS Cluster, CNI, and ContainerRegistryReadOnly.
- Add-ons (addons.tf): AWS EBS CSI driver (for gp3 volumes) and Kubernetes Metrics Server (for HPA).

Cost Safety & Teardown Runbook

- Reproducibility: Configured via terraform.tfvars.example with parameterized variables.
- Automated Teardown: Documented runbook in terraform/README.md for clean resource destruction.
- Zero-Leak Guarantee: PVCs and Ingresses uninstalled prior to terraform destroy to prevent dangling ALBs.

Kubernetes Packaging: Helm & Workloads

Declarative Packaging, Non-Root Workloads, and Migration Job Hooks

Workload Declarations

- Backend Deployment: 2 replicas, rolling updates (maxSurge: 1, maxUnavailable: 0), liveness and readiness probes.
- Frontend Deployment: 2 replicas, unprivileged Nginx reverse-proxying API traffic to backend.
- PostgreSQL StatefulSet: 1 replica with gp3 StorageClass PersistentVolumeClaim (10Gi).
- PodDisruptionBudget (PDB): Guarantees at least 1 backend pod remains available during node maintenance.

SecurityContext & Hardening

- Backend Pods: runAsNonRoot: true, runAsUser: 10001, runAsGroup: 10001, drop ALL capabilities.
- Frontend Pods: runAsNonRoot: true, runAsUser: 101, runAsGroup: 101, drop ALL capabilities.
- Ingress Controller: Routes / to frontend and /api, /health, /ready, /metrics to backend service.

Database Migration Hook (Race Prevention)

- Problem Solved: Multiple backend replicas starting simultaneously cause database migration lock races.
- Helm Hook Job: migration-job.yaml runs 'alembic upgrade head' as a pre-install/pre-upgrade hook.
- App pods start with RUN_MIGRATIONS=false only AFTER the migration Job reports success.

GitOps with ArgoCD

Continuous Reconciliation, Automated Syncing, and Self-Healing Drift Control

ArgoCD Application Configuration

- Manifest: argocd/application.yaml targeting namespace podiumhq with project: default.
- Source: Watches this Git repository (path: helm/podiumhq, targetRevision: HEAD).
- Environment Values: Uses values-prod.yaml for production cluster deployment.

Automated Sync & Self-Healing

- Automated Sync: ArgoCD automatically detects new image tag commits and applies them without human intervention.
- Self-Healing (selfHeal: true): If an engineer manually alters cluster state via kubectl, ArgoCD immediately reverts the drift.
- Pruning (prune: true): Deletes orphaned cluster resources when removed from Helm templates.

Live Drift Demonstration (Viva Check)

- Step 1: Manually scale backend deployment: 'kubectl scale deployment podiumhq-backend --replicas=5'.
- Step 2: ArgoCD detects state drift and marks application OutOfSync.
- Step 3: Within seconds, ArgoCD self-heals the cluster back to the Git source of truth (2 replicas).

Observability: Prometheus & Grafana

Full-Stack System Metrics, Custom Domain Telemetry, and Alerting Rules

Exposed Telemetry (/metrics)

- Standard Metrics: HTTP request counts, response status codes (2xx/4xx/5xx), and latency histograms.
- Runtime Telemetry: Python garbage collection, memory usage, CPU seconds, and active file descriptors.
- Instrumentation: Powered by prometheus-client and prometheus-fastapi-instrumentator.

Custom Domain Business Metrics

- podiumhq_scores_submitted_total: Counter tracking total judge scores submitted across rounds.
- podiumhq_teams_by_status: Gauge tracking team counts categorized by lifecycle status (ACTIVE, EVALUATED).
- podiumhq_leaderboard_requests_total: Counter tracking traffic to the live standings endpoint.

Grafana Dashboard & PrometheusRules

- Grafana JSON (monitoring/grafana-dashboard.json): 5+ real-time panels tracking RPS, p95 latency, errors, and domain stats.
- PrometheusRule (monitoring/alerts.yaml): Automated alerts for High Error Rate (>1% 5xx) and Backend Target Down.
- ServiceMonitor: Scrapes backend service every 15s via Prometheus Operator.

Testing, Security, & Real Challenges

91% Test Coverage, Container Non-Root Verification, and Practical Bug Fixes

Automated Testing Verification

- Pytest Results: 31 passed in 1.32s with 90.99% overall code coverage (target: >=80%).
- Test Isolation: SQLite with StaticPool ensures zero interference between test runs.
- Business Logic Verified: Tie-breaking, duplicate score rejection, and round lifecycle locks.

Container Security Verification

- Backend: Verified non-root execution via 'docker compose exec backend id' -> uid=10001(appuser).
- Frontend: Verified unprivileged execution via 'docker compose exec frontend id' -> uid=101(nginx).
- Secret Scanning: Gitleaks scan confirmed zero committed credentials or API keys in Git history.

Real Implementation Challenges Solved

- Challenge 1: In-container SQLite coverage database was blocked because /app was owned by root. Fixed by chowning /app to appuser.
- Challenge 2: Frontend healthcheck failed with connection refused because BusyBox wget resolved localhost to IPv6 ::1. Fixed by adding listen [::]:8080.
- Challenge 3: Test dependencies were excluded by .dockerignore. Fixed by selectively including tests for in-container validation.

Key Learnings & Conclusion

100-Point Rubric Fulfillment, Production Takeaways, and Future Roadmap

Rubric Self-Audit (RUBRIC_CHECK.md)

- 100% compliance across all 10 modules (M1 Application to M10 Presentation).
- Locally runnable components (Docker Compose, Pytest 91%, Non-Root, Probes) are 100% verified green.
- Cloud-only components (AWS EKS apply, live ArgoCD) are fully written, validated, and documented.

Key Engineering Lessons

- Pre-upgrade Helm hooks are essential to avoid database schema race conditions across replicated pods.
- Dual-stack IPv4/IPv6 awareness is critical when containerizing unprivileged Nginx on modern Alpine Linux.
- Immutable image tagging by Git commit SHA ensures reliable GitOps rollback without caching ambiguities.

Future Production Enhancements

- Transition in-cluster PostgreSQL to Amazon Aurora Multi-AZ with automated automated failover.
- Implement OpenTelemetry distributed tracing with AWS X-Ray for end-to-end request tracing.
- Deploy Istio Service Mesh with mTLS and fine-grained authorization policies across microservices.